HOTEL BOOKING SYSTEM

Team - 14

TEAM INTRODUCTION

- Girish Kumar - 2510030019(File Handling,Hash Table,Queue)
- Hasini M-2510030098(Arrays,Binary)
- Yogendra-2510030147(Linear,Bubble sort)
- Sirisha-2510030217(Stacks,Quick)
- Granth. J-2510030181 (Insertion,Linked List)

ABSTRACT

The Hotel Booking Management System is a Java-based application developed to efficiently manage hotel reservations using multiple data structures and algorithms. The system enables users to add, display, search, sort, and manage customer booking records while ensuring optimized performance. A Linked List is used for dynamic storage of records, a Queue handles booking order, a Priority Queue processes urgent bookings first, and a Stack provides undo functionality. A Hash Table is implemented for fast room lookup, while arrays are used for room data management. The system incorporates sorting techniques such as Bubble Sort, and Linear Search for efficient data retrieval. File handling is used to save and load data, ensuring persistence. Overall, the project demonstrates the practical implementation of data structures in solving real-world problems by building an efficient, scalable, and user-friendly hotel booking system.

MODULE

Record Management (Linked List)
Queue (Booking Order)
Priority Queue (Urgent Handling)
Stack (Undo Feature)
Hash Table (Fast Search)
Sorting & Searching Algorithms
File Handling (Data Storage)

MODULE DESCRIPTION

1. **Record Management Module (Linked List)**
- Stores customer booking records dynamically using a linked list.
- Allows easy insertion and traversal of records.
- Topic Used: Linked List for dynamic memory allocation and flexible data handling.

2. Booking Queue Module (Queue)
- Manages booking requests in First-In-First-Out (FIFO) order.
- Ensures fair processing of customers.
- Topic Used: Queue data structure for sequential processing.

3. Priority Handling Module (Priority Queue)
- Handles urgent bookings before normal bookings.
- Uses priority-based processing for efficiency.
- Topic Used: Priority Queue (Heap) for priority scheduling.

4. **Undo Operation Module (Stack)**
- Allows users to undo the most recently added booking.
- Works on Last-In-First-Out (LIFO) principle.
- Topic Used: Stack for reversing recent operations.

5. **Room Management Module (Array)**
- Stores room details like number, rating, price, and availability.
- Provides simple and direct access to room data.
- Topic Used: Arrays for static data storage.

6. **Search Module (Searching Algorithms)**
- Enables searching by ID using Binary Search and by name using Linear Search.
- Improves efficiency in locating records quickly.
- Topic Used: Searching algorithms for fast data retrieval.

7. Sorting Module (Sorting Algorithms)

- Sorts customer records based on ID.
- Implements Bubble Sort, Insertion Sort, and Quick Sort.
- Topic Used: Sorting algorithms for organized data management.

8**. Hashing Module (Hash Table)**

- Provides fast lookup of room numbers.
- Uses hashing technique with linear probing.
- Topic Used: Hash Table for efficient search operations.

9**. File Handling Module (File I/O)**

- Saves and loads customer and room data from files.
- Ensures data persistence after program execution.
- Topic Used: File handling using Java I/O streams.

CODE INPUT

```java
class Record {
    int customerID;
    String name;
    int roomNo;
    String checkIn;
    String checkOut;
    double payment;
    boolean urgent;
    Record link;

    Record(int id, String n, int room, String in, String out, double pay, boolean
        customerID = id;
        name = n;
        roomNo = room;
        checkIn = in;
        checkOut = out;
        payment = pay;
        urgent = urg;
        link = null;
    }
}


class records {
    Record head;

    void addRecord(Record r) {
        if (head == null) {
            head = r;
        } else {
            Record temp = head;
            while (temp.link != null) {
                temp = temp.link;
            }
            temp.link = r;
        }
    }
```

```java
boolean flag = true;
while (flag) {
    System.out.println("----🏨Welcome to KL Hotel Booking:😊----");
    System.out.println("1. Add Record 😊");
    System.out.println("2. Display all records");
    System.out.println("3. Add new room details (for hotel owner)");
    System.out.println("4. Search for a record of customer");
    System.out.println("5. Sort all records");
    System.out.println("6. Undo Recently added record");
    System.out.println("7. Do priority order first");
    System.out.println("8. Show all rooms");
    System.out.println("9. ❌ Exit");
    System.out.println();
    System.out.print("Enter Choice:");

    int ch = sc.nextInt();
    sc.nextLine();
    switch(ch) {
    case 1:

        System.out.print("Enter ID: ");
        int id = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Name: ");
        String name = sc.nextLine();
        System.out.print("Enter Room No: ");
        int room = sc.nextInt();
        sc.nextLine();
        System.out.print("Enter Check-in: ");
        String ci = sc.nextLine();
        System.out.print("Enter Check-out: ");
        String co = sc.nextLine();
        System.out.print("Enter Payment: ");
        double pay = sc.nextDouble();
        sc.nextLine();
        System.out.print("Is urgent? (true/false): ");
        boolean urg = sc.nextBoolean();
        Record r = new Record(id, name, room, ci, co, pay, urg);
        records oi=new records();
        oi.addRecord(r);
```

```java
static void bubbleSort(Record[] arr) {
    for (int i = 0; i < arr.length - 1; i++) {
        for (int j = 0; j < arr.length - i - 1; j++) {
            if (arr[j].customerID > arr[j + 1].customerID) {
                Record temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

static void insertionSort(Record[] arr) {
    for (int i = 1; i < arr.length; i++) {
        Record key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j].customerID > key.customerID) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

static void quickSort(Record[] arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

CODE OUTPUT

```
----✌Welcome to KL Hotel Booking:😉----
1. Add Record 😊
2. Display all records
3. Add new room details (for hotel owner)
4. Search for a record of customer
5. Sort all records
6. Undo Recently added record
7. Do priority order first
8. Show all rooms
9. ✖ Exit

Enter Choice:1
Enter ID: 201
Enter Name: girish
Enter Room No: 302
Enter Check-in: 29-03-2026
Enter Check-out: 30-03-2026
Enter Payment: 1200
Is urgent? (true/false): true
Record added.
```


```
----✌Welcome to KL Hotel Booking:😉----
1. Add Record 😊
2. Display all records
3. Add new room details (for hotel owner)
4. Search for a record of customer
5. Sort all records
6. Undo Recently added record
7. Do priority order first
8. Show all rooms
9. ✖ Exit
```


```
Enter Choice:3
Enter management id: klhadmin
Enter password: 123456
Wrong password
```

OUTCOMES-WHAT WE HAVE LEARNT

1. **Real-World Use of Data Structures**
Applied Linked List, Queue, Stack, Priority Queue, and Hash Table to solve a practical booking problem efficiently.

2. **Efficient Data Handling**
Learned how to organize, store, and retrieve data using searching and sorting algorithms.

3. **System Design Thinking**
Understood how to break a large problem into modules like booking, search, and management.

4. **Optimization Techniques**
Used appropriate data structures to improve performance (e.g., priority queue for urgent tasks).

OUTCOMES-WHAT WE HAVE LEARNT

5**. File Handling Skills**
Implemented file operations to save and load data, ensuring persistence.

6. **Debugging and Logic Building**
Improved ability to identify errors and build correct logical flow in programs.

7. **Practical Development Experience**
Gained confidence in building a complete working system and presenting it effectively.

FUTURE ENHANCEMENT

1. Database Integration
Replace file handling with MySQL or MongoDB for better data management, scalability, and security.

2. Online Booking System
Convert into a web application using HTML, CSS, JavaScript, and backend frameworks like Spring Boot.

3. Payment Gateway Integration
Add secure online payments using APIs like Razorpay or Stripe.

4. Real-Time Room Availability
Implement live tracking using database queries and synchronization techniques.

5. Mobile Application
Build an Android app using Java/Kotlin or Flutter for easy access and better user experience.

THANK YOU